

Семинар №7

Move-семантика

Мотивация

- До сих пор мы умели только копировать объекты или использовать ссылки на них
- Но часто хотелось бы перемещать объекты
- Например, если мы знаем, что объект нам “больше не нужен”, мы можем отдать ресурсы, которыми он владеет
- Введем операцию перемещения объекта – мы забираем ресурсы у одного объекта и отдаём другому

Rule of ...

- Three: If a class requires a user-defined **destructor**, a user-defined **copy constructor**, or a user-defined **copy assignment operator**, it almost certainly requires all three.
- Five: If a class requires a user-defined **destructor**, a user-defined **copy constructor**, a user-defined **copy assignment operator**, a user-defined **move constructor**, a user-defined **move assignment operator**, it almost certainly requires all five.

Move-constructor and assignment operator

- `Test(Test&& other)` – move constructor
- `Test& operator=(Test&& other)` – move assignment operator
- Так как мы хотим забрать владение ресурсами у `other`, принимаем его не через `const`
- Двойная ссылка (`&&`) требует, чтобы `other` можно было “разобрать на запчасти”
- У разрушаемого объекта нужно не забыть занулить указатель (если таковой имеется), чтобы избежать `double free` в деструкторе
- N.B.! Разрушенный объект должен быть в полностью корректном состоянии с точки зрения состояния объекта, таким объектом можно пользоваться позже, просто он не владеет ресурсами

std::move

- Хотим пометить объект как такой, что его “можно разбирать на запчасти”
- **N.B.!** Сам std::move никак не меняет сам объект!
- Но использование его в выражении с std::move оставляет пометку
- Таким образом, эта пометка есть только в compile time, но не в run time
- Неявный каст к T&& не происходит, иначе все бы мувалось, даже когда мы этого не хотим

Глобальная идея

- Перемещение – это важная фундаментальная особенность языка
- Некоторые классы умеют только перемещаться, но не копироваться (`std::unique_ptr`, `std::mutex`, ...)
- В Rust по умолчанию происходит перемещение объекта, для создания копии нужно явно этого попросить, потому что в большинстве случаев при написании “=” требуется именно перемещение
- Однако, в Rust запрещено пользоваться перемещенным объектом, это поддерживается компилятором

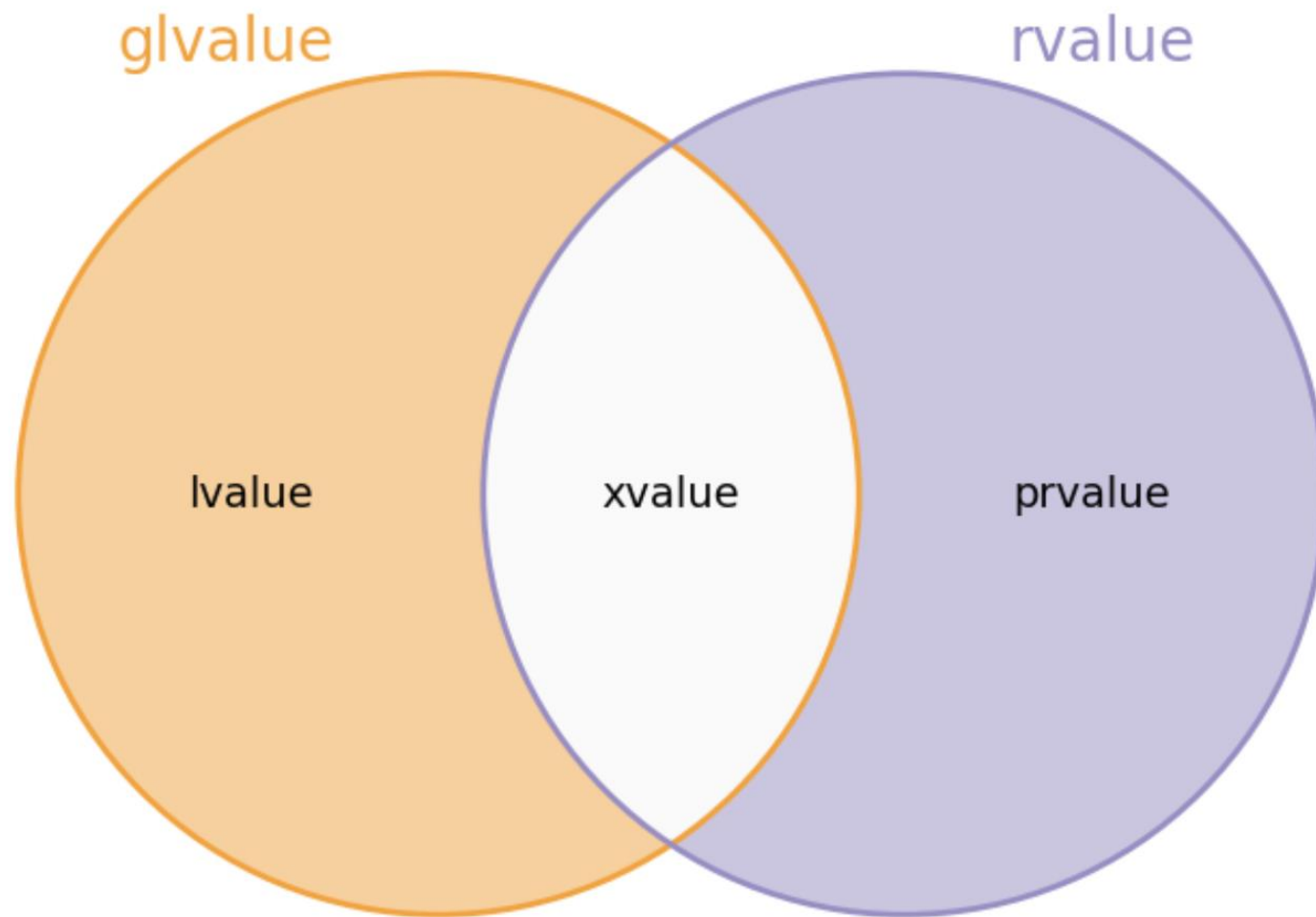
Обратная совместимость

- Если у класса явно определен конструктор копирования, но не конструктор перемещения, то при попытке перемещения будет вызван конструктор копирования
- Аналогичное верно для оператора присваивания
- Это правило обеспечивает корректность работы для классов без поддержки move-семантики
- Если же все из rule of three дефолтное, то конструктор копирования / оператор присваивания копированием будут дефолтными (перемещают поля)

lvalue & rvalue

- [Value categories](#)
- Определены для **выражений** (expressions)
- lvalue: variable name, function name, T& foo, ...
- rvalue: literals, T foo, T&& foo, ...

Value categories



Value Category

Rvalue references

- T& - lvalue-reference
- T&& - rvalue-reference
- При возврате из функции rvalue-ref является rvalue
- rvalue-ref может быть проинициализирована только rvalue
- rvalue-ref продляют жизнь объектам (как и const lvalue-ref)
- Константность не должна нарушаться как с lvalue, так и с rvalue, причем навешивать константность можно, а убирать только явным кастом
- `const T& + std::move -> const T&& => foo(const T&), но не foo(T&&)`

Reference qualifiers

- Перегрузка по value category у объекта, от которого вызываем метод: `&` / `&&`
- Можно перегружать одновременно как по `const`, так и по `reference qualifiers`
- `const & qualifiers` позволяют принять себя по `rvalue ref` (т.е. как и раньше можем проинициализировать `const lvalue ref` с помощью `rvalue ref`)
- Чиним “`a + b = 42`” – оператор присваивания должен иметь `& qualifier`